What's new

Decomposing a factorial into large factors (second version)

Terence Tao · 3 June 2025

https://terrytao.wordpress.com/2025/06/03/decomposing-a-factorial-into-large-factors-second-version/

tao2typ · converted to Typst 2026-10-14 (UTC)

Boris Alexeev, Evan Conway, Matthieu Rosenfeld, Andrew Sutherland, Markus Uhr, Kevin Ventullo, and I have uploaded to the arXiv a second version of our paper "Decomposing a factorial into large factors". This is a completely rewritten and expanded version of a previous paper of the same name. Thanks to many additional theoretical and numerical contributors from the other coauthors, we now have much more precise control on the main quantity $t(N)$ studied in this paper, allowing us to settle all the previous conjectures about this quantity in the literature.

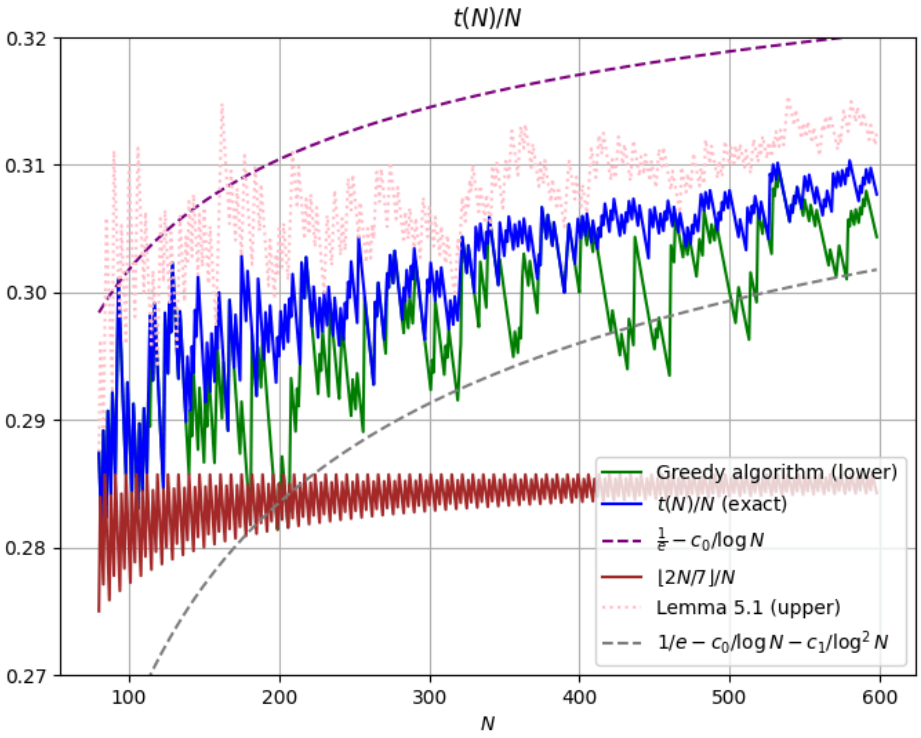
As discussed in the previous post, $t(N)$ denotes the largest integer t such that the factorial $N!$ can be expressed as a product of N factors, each of which is at least t. Computing $t(N)$ is a special case of the bin covering problem, which is known to be NP-hard in general; and prior to our work, $t(N)$ was only computed for $N \leq 599$; we have been able to compute $t(N)$ for all $N \leq 10000$. In fact, we can get surprisingly sharp upper and lower bounds on $t(N)$ for much larger N, with a precise asymptotic

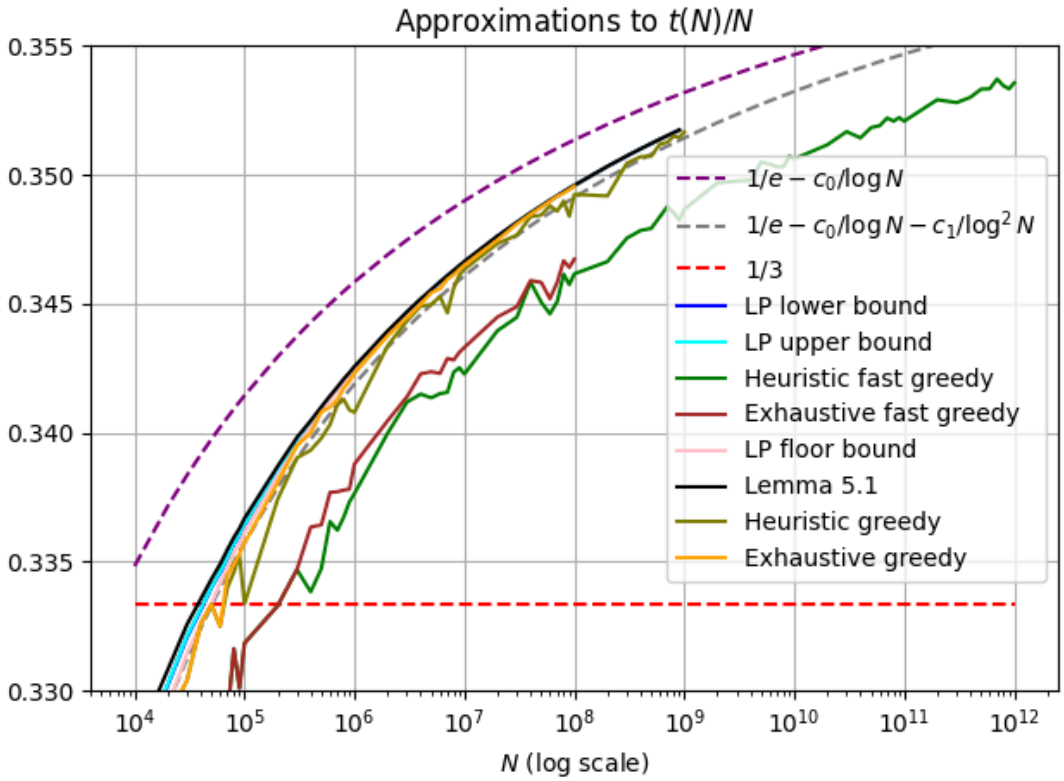
$$\frac{t(N)}{N} = \frac{1}{e} - \frac{c_0}{\log N} - \frac{O(1)}{\log^{1+c} N}$$

for an explicit constant $c_0 = 0.30441901\ldots$, which we conjecture to be improvable to

$$\frac{t(N)}{N} = \frac{1}{e} - \frac{c_0}{\log N} - \frac{c_1 + o(1)}{\log^2 N}$$

for an explicit constant $c_1 = 0.75554808\ldots$: ... For instance, we can demonstrate numerically that $0 \leq t(9 \times 10^8) - 316560601 \leq 113$.

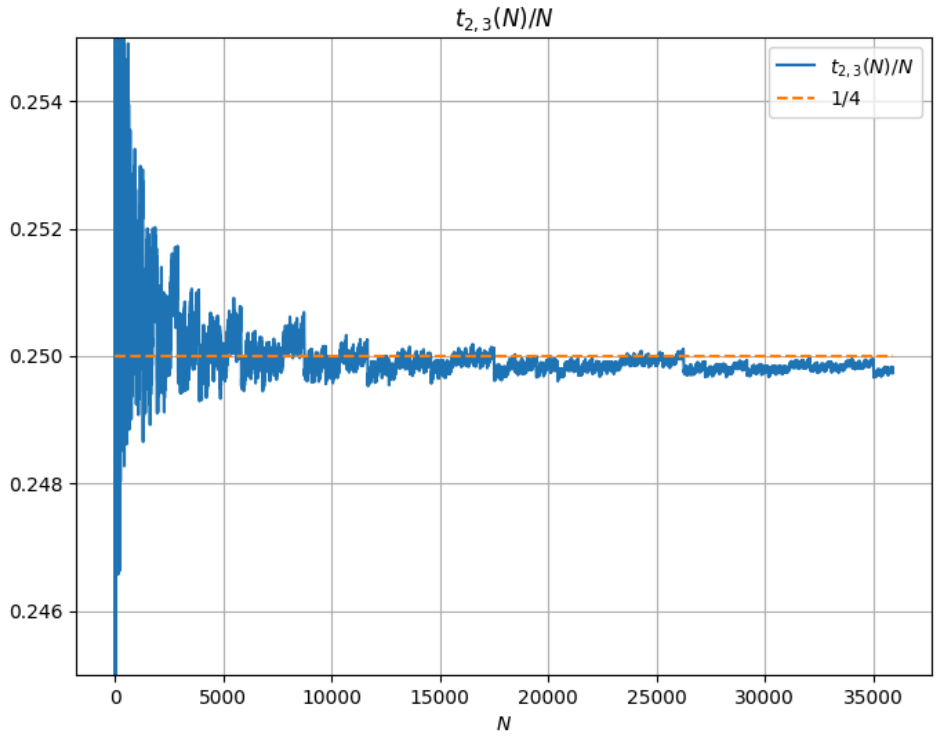




As a consequence of this precision, we can verify several conjectures of Guy and Selfridge, namely

- $t(N) \leq N/e$ for all $N \neq 1, 2, 4$.
- $t(N) \geq \lfloor 2N/7 \rfloor$ for all $N \neq 56$.
- $t(N) \geq N/3$ for all $N \geq 3 \times 10^5$. (In fact we show this is true for $N \geq 43632$, and that this threshold is best possible.)

Guy and Selfridge also claimed that one can establish $t(N) \geq N/4$ for all large N purely by rearranging factors of 2 and 3 from the standard factorization $1 \times 2 \times \ldots \times N$ of $N!$, but surprisingly we found that this claim (barely) fails for all $N > 26244$:



The accuracy of our bounds comes from several techniques:

- *Greedy algorithms*, in which one allocates the largest prime factors of $N!$ first and then moves to smaller primes, provide quickly computable, though suboptimal, lower bounds on $t(N)$ for small, medium, and moderately large values;
- *Linear programming* and *integer programming* methods provides extremely accurate upper and lower bounds on $t(N)$ for small and medium values of N;
- *Rearrangement methods* can be analyzed asymptotically via linear programming, and work well for large N; and
- The *modified approximate factorization* strategy, discussed in the previous post is now sharpened by using 3-smooth numbers (products of 2 and 3) as the primary "liquidity pool" to reallocate factors of $N!$, as opposed to the previous approach of only using powers of 2.

To me, the biggest surprise was just how stunningly accurate the linear programming methods were; the very large number of repeated prime factors here actually make this discrete problem behave rather like a continuous one.

Comments (14)

dcuzz · 4 June 2025

not sure if it's my device but some of these formulas don't look right (wrt formatting, not content)

Anonymous · 4 June 2025

FYI, the math after the explicit constant c_1 does not parse and the display environment before it has an error with the fraction and the equals sign.
[Fixed, thanks – T]

> **Anonymous** · 4 June 2025
>
> The math parses now but TWO display environments still look wrong to me. The thing I see is that there is a frac with t(N) on top and an equals sign on the bottom. So, I think there is an N missing in the bottom of two fracs in display environments: I think there should be frac{t(N)}N strictly before the = sign.
> *[Hopefully fixed now – T.]*
>
> > **Anonymous** · 4 June 2025
> >
> > For the display environment following "conjecture to be improvable", I think the exponent 1+c should be 2.
> > *[Corrected, thanks -T.]*

Anonymous · 4 June 2025

The formulas in this post don't seem to be rendering correctly. Not sure if it's just me.
[This appeared to be some sort of temporary caching issue. -T]

Anonymous · 4 June 2025

This reminds me about a similar problem with a fixed number of factors (independent of N). E.g., for 3 factors we get https://oeis.org/A061030

> **Terence Tao** · 4 June 2025
>
> I would imagine that the linear programming/integer programming method would be quite effective at this problem also. For any fixed number k of factors, one should be able to get very close to the trivial upper bound of $(N!)^{1/k}$, though I do not have a good guess for now as to what the size of the error term should be.
>
> > **Anonymous** · 4 June 2025
> >
> > Getting exact solutions with ILP is tricky as with growth of N it becomes either inaccurate or slow. I've brought this issue to discussion at https://mathoverflow.net/q/419722
> >
> > > **Boris Alexeev** · 4 June 2025
> > >
> > > The (integer) linear programs in this paper are much better behaved than the ones in that MathOverflow question. In our case, if for example you have $N = 43631$ and we wish to show that $N!$ cannot be factored into N factors each of which is at least $T = \lceil N/3 \rceil = 14544$, we use variables $m_T, ldots, m_N$ representing how many times we use each factor T through N. There are "budget" constraints for each prime p that say $sum_j m_j \nu_p(j) \leq \nu_p(N!)$. We can either add the constraint that we want there to be at least $\sum_j m_j \geq N$ factors and solve a feasibility problem — or alternatively, we can set the optimization

objective to maximize $\sum_j m_j$ and then compare this objective to N. Note that all of the coefficients are small integers, often 0 or 1.
The optima also behave quite well for N in this range. In the example I gave above, there is no need to use integer programming: the LP relaxation is already tight enough. And in order to be sure, we verified the dual certificate in exact arithmetic.
For comparison, if $N = 20$ in the MO post above, we seek to factor 20! into two terms each of which is as close to $\sqrt{20!} \approx 1.5 \times 10^9$ as possible. The natural integer programming formulation has many coefficients of the form $\log p$, which aren't even rational (yet alone small integers). Unfortunately, it is not effective to use our formulation instead because then there would be an enormous number of variables m_j for different j (and the approach would be worse than a simple brute-force loop).
I think the major difference is that in our formulation, the resulting answer T is of the same order as N, while when splitting into a constant number of factors, it becomes enormous.



Boris Alexeev · 5 June 2025

A couple more thoughts regarding the two different programs:
For the program in the MO question, the linear program relaxation (ie, not *integer*) has the obvious bound as a solution, namely that each factor must be at most $\sqrt{N!}$. In our case, the obvious bound $(N!)^{1/N}$ is a *feasible* dual solution, but not an optimal one. And in fact some of the paper finds better dual solutions purely mathematically (as opposed to computationally) and these rigorously prove upper bounds tighter than the obvious one.
I will also note that the *specific* (and surprising!) issue that arose in the MO question, namely that the LP solver decides an infeasible solution is feasible due to its rounding tolerance, cannot happen in our setting. That is because all of the coefficients and variables are (small) integers, and there's no rounding issue verifying it! Besides the separate ways in which we verify our LP results, this is a concrete sense in which our LPs are more well-behaved.
But thank you very much for linking to the MO question! I use linear programs often in my research (and then almost always verify the results in another way), and it's nice to know which particular errors can arise. That one is particularly glaring.



The Guy-Selfridge Conjecture – Kevin Ventullo · 4 June 2025

[…] was a fun project! I encourage interested readers to check out the original post, as well as the follow-up post discussing the improved […]



Anonymous · 7 June 2025

"products of 2 and 3" -> "products of powers of 2 and 3"?
[Here I am implicitly permitting repetitions of factors. -T]

Stefan Persich · 26 June 2025

Really enjoying this line of work – especially how the asymptotic behavior around the N/e barrier is emerging from increasingly refined constructions.
One potentially useful variation: rather than factoring `N!` globally, have you ever tried splitting it into multiplicative blocks of size about `N / log N`, and within each block, construct products from large factors only (say ≥ `sqrt(N)`), optimizing locally. This seems to reduce the rounding inefficiencies that arise in the greedy 2–3 rearrangements.

In my simulations up to `N = 20,000`, this block-based method consistently improves the total count `t(N)` by around **0.5–1.5%**, with better scaling as `N` increases. It might offer a cleaner asymptotic route toward saturating the `N/e` limit.
Would be interesting to hear if you've explored something similar—or if this strategy could blend with your entropy-based or rearrangement heuristics.



Growth rates of sequences governed by the squarefree properties of its translates | What's new · 1 December 2025

[…] However, we do not currently have enough numerical data for the sequence to completely confirm the conjecture in all cases. This could potentially be a crowdsourced project (similar to the Erdős-Guy-Selfridge project reported on in this previous blog post). […]